

Carbon Footprint Tracker — Core Architecture

1. Data Schema & State Management

LocalStorage is the right call here, not IndexedDB — your data is small, flat, doesn't need transactions or indexes, and avoids the async API overhead and boilerplate IndexedDB demands. IndexedDB only earns its weight once you're past a few thousand structured records or need offline querying; a daily log app for one user will sit comfortably under LocalStorage's ~5-10MB ceiling for years.

Single key, single JSON blob — this avoids the cost of multiple `getItem/setItem` round-trips and keeps reads/writes atomic (no partial-write corruption between related fields).

javascript

```
// localStorage key: 'cft_state'
{
  "version": 1, // schema version for safe future migrations
  "streak": {
    "current": 4,
    "longest": 11,
    "lastLogDate": "2026-06-17" // ISO date string, used to validate streak continuity
  },
  "totals": {
    "lifetimeKgCO2e": 842.7,
    "lifetimeKgSaved": 0, // populated once a baseline/average is established
    "daysLogged": 58
  },
  "logs": {
    // keyed by ISO date string for O(1) lookup/overwrite, no array scanning needed
    "2026-06-17": {
      "transportation": { "gas_car": 8, "bus": 0, "train": 0, "ev": 0, "walking": 0, "cycling": 0 },
      "diet": { "beef": 0, "chicken": 1, "fish": 0, "vegetarian": 1, "vegan": 0 },
      "energy": {
        "streaming_video": 2,
        "laundry_hot_wash_dry": 0,
        "laundry_cold_wash_air_dry": 1,
        "room_light_led": 5,
        "room_light_incandescent": 0
      },
      "totalKgCO2e": 5.84, // cached result, avoids recomputation on every render
      "dominantCategory": "transportation" // cached for insight lookups
    }
  }
}
```

Design rationale:

- **Object-keyed logs** (`logs[ "2026-06-17" ]`) instead of an array — avoids `.find().filter()` scans as history grows; lookups, edits, and overwrites are O(1).
 - **Cached `totalKgCO2e` and `dominantCategory`** per day — the engine computes once on save, not on every app load or chart render.
 - **Flat factor categories matching your JSON exactly** — zero translation layer between your provided data and stored logs, so the engine can iterate `Object.entries()` directly against either structure.
 - **Pruning strategy**: if you want to cap storage growth, keep raw daily logs for a rolling 90 days and roll older entries into monthly aggregates (`totals.monthlyArchive`). Not included below to keep the engine minimal, but the schema's flat-key design makes this trivial to bolt on later.
-

2. The **CarbonEngine** Class

javascript

```
/**
 * CarbonEngine
 * Pure computation layer — no DOM access, no storage access.
 * Takes data in, returns data out. Fully unit-testable in isolation.
 */
class CarbonEngine {
  /**
   * @param {Object} emissionFactors - baseline_emission_factors_kg_co2e object
   * @param {Array} nudges - behavioral_nudges array
   * @param {Object} equivalents - relatable_equivalents_per_1_kg_co2e object
   */
  constructor(emissionFactors, nudges, equivalents) {
    this.factors = emissionFactors;
    this.equivalents = equivalents;

    // Map nudges by category once at construction — O(1) lookup at runtime
    // instead of Array.find() scanning on every insight request.
    this.nudgeMap = this._buildNudgeMap(nudges);
  }

  /**
   * Maps each emission category to its most relevant nudge.
   * Explicit mapping (not string-matching) avoids fragile coupling
   * to nudge wording, which can change independently of category keys.
   */
  _buildNudgeMap(nudges) {
    const findNudge = (action) => {
      const match = nudges.find((n) => n.action === action);
    }
  }
}
```

```

    return match ? match.personalized_insight : null;
  };

  return {
    transportation_gas_car: findNudge('Active Transportation'),
    transportation_bus: findNudge('Public Transit Shift'),
    transportation_train: findNudge('Public Transit Shift'),
    diet_beef: findNudge('Meatless Substitution'),
    diet_chicken: findNudge('Meatless Substitution'),
    diet_fish: findNudge('Meatless Substitution'),
    energy_laundry_hot_wash_dry: findNudge('Cold Wash & Line Dry'),
    energy_streaming_video: findNudge('Digital Efficiency'),
    energy_room_light_incandescent: findNudge('LED Optimization'),
    // fallback for low-impact categories with no dedicated high-leverage nudge
    default: findNudge('Eliminate Phantom Loads'),
  };
}

/**
 * Calculates total daily footprint and per-category subtotals.
 * @param {Object} dailyLog - { transportation: {km...}, diet: {servings...}, energy: {hours...} }
 * @returns {{ totalKgCO2e: number, breakdown: Object, dominantCategory: string|null,
dominantKey: string|null }}
 */
calculateDailyFootprint(dailyLog) {
  const breakdown = { transportation: 0, diet: 0, energy: 0 };
  let dominantKey = null; // e.g. "transportation_gas_car" — most specific emitter
  let dominantValue = 0;

  const categoryFactorMap = {
    transportation: this.factors.transportation_per_km,
    diet: this.factors.diet_per_serving,
    energy: this.factors.energy_and_tech_per_hour,
  };

  for (const category of Object.keys(categoryFactorMap)) {
    const userEntries = dailyLog[category];
    const factorSet = categoryFactorMap[category];
    if (!userEntries) continue;

    for (const [activity, amount] of Object.entries(userEntries)) {
      const factor = factorSet[activity];
      // Guard against malformed input keys (typos, future schema drift)
      if (typeof factor !== 'number' || typeof amount !== 'number' || amount <= 0) {
        continue;
      }

      const emission = factor * amount;

```

```

    breakdown[category] += emission;

    // Track the single highest-emitting activity across ALL categories,
    // not just the highest category — this is what makes the nudge specific.
    if (emission > dominantValue) {
        dominantValue = emission;
        dominantKey = `${category}_${activity}`;
    }
}
}

const totalKgCO2e = Object.values(breakdown).reduce((sum, v) => sum + v, 0);

return {
    totalKgCO2e: this._round(totalKgCO2e),
    breakdown: {
        transportation: this._round(breakdown.transportation),
        diet: this._round(breakdown.diet),
        energy: this._round(breakdown.energy),
    },
    dominantKey, // e.g. "transportation_gas_car"
    dominantCategory: dominantKey ? dominantKey.split('_')[0] : null,
};
}

/**
 * Returns the single most relevant behavioral nudge for the day,
 * based on the highest-emitting specific activity (not just category).
 * @param {Object} dailyLog
 * @returns {string} personalized insight text
 */
getPersonalizedInsight(dailyLog) {
    const { dominantKey } = this.calculateDailyFootprint(dailyLog);

    if (!dominantKey) {
        return this.nudgeMap.default || 'Log an activity to get a personalized insight.';
    }

    return this.nudgeMap[dominantKey] || this.nudgeMap.default;
}

/**
 * Converts a kg CO2e value (typically "saved") into relatable equivalents.
 * @param {number} totalKgSaved
 * @returns {{ smartphoneCharges: number, milesDrivenGasCar: number,
    daysTreeAbsorption: number }}
 */
getRelatableEquivalent(totalKgSaved) {

```

```

if (typeof totalKgSaved !== 'number' || totalKgSaved <= 0) {
  return { smartphoneCharges: 0, milesDrivenGasCar: 0, daysTreeAbsorption: 0 };
}

return {
  smartphoneCharges: this._round(totalKgSaved * this.equivalents.smartphone_charges),
  milesDrivenGasCar: this._round(totalKgSaved * this.equivalents.miles_driven_gas_car),
  daysTreeAbsorption: this._round(totalKgSaved * this.equivalents.days_tree_absorption),
};
}

/** Rounds to 2 decimal places — avoids floating point noise (e.g. 5.8400000000001) */
_round(num) {
  return Math.round(num * 100) / 100;
}
}

```

Key engineering decisions:

- **Constructor takes raw data, not a fetch call** — the engine has zero knowledge of where the JSON comes from (inline script, fetch, import). Keeps it framework-agnostic and trivially testable with `new CarbonEngine(mockFactors, mockNudges, mockEquivalents)`.
- **Nudge resolution is activity-level, not category-level.** A naive version would say "your diet category is highest, show the diet nudge." This version finds the single highest-emitting *specific activity* (e.g., `diet_beef` vs `diet_chicken`) so a user who only ate chicken doesn't get told to stop eating beef. This is the difference between a generic and a genuinely personalized nudge.
- **Defensive iteration** — `typeof factor !== 'number'` guards mean a malformed log object (bad key, string instead of number) silently skips rather than throwing or producing `NaN` that poisons the total. This matters because `LocalStorage` data can be hand-edited or corrupted by a bad migration.
- **No mutation, no side effects** — every method is a pure function of its inputs. State management (reading/writing `LocalStorage`, updating streaks) belongs in a separate thin controller layer, not in the engine. This separation is what makes the engine reusable if you ever add a backend.

3. Architecture Blueprint for a Sub-10MB Build

```

/carbon-tracker
├── index.html          # single entry point, no bundler needed
├── /src
│   ├── /core
│   │   ├── CarbonEngine.js  # pure logic, zero DOM/storage imports
│   │   └── constants.js     # the JSON factors/nudges/equivalents, as ES module exports
│   └── /data

```



Size-guaranteeing decisions:

1. **No bundler, no build step.** Use native ES modules (`<script type="module">`) and `import/export` directly. Bundlers (Webpack, Vite, Rollup) pull in node_modules and config overhead that's pointless at this scale — modern browsers handle native module graphs fine for an app this size, and `/dist` becomes a straight copy of `/src` with no transpilation artifacts bloating it.
2. **Zero npm dependencies.** No date library (use native `Date + toISOString().split('T')[0]` for day keys), no state library (a single `LocalStorage` blob *is* your state container), no chart library unless absolutely necessary (hand-rolled `<canvas>` or SVG bars cost kilobytes; Chart.js costs hundreds of KB).
3. **Constants as a code module, not a fetched JSON file.** Importing `constants.js` as an ES module lets the minifier tree-shake and inline it into the final bundle with zero extra HTTP request and no JSON-parse runtime cost. A separate `.json` fetched at runtime adds a network round-trip and async complexity for data that never changes.
4. **One minification pass, no source maps in `/dist`.** Run `esbuild` (a single ~10MB dev dependency, never shipped) as a one-line CLI command to minify `src/` into `dist/bundle.js`. No need for Babel or polyfills if you target evergreen browsers only.
5. **Audit images/fonts separately** — they're typically the actual size offender, not JS. SVG icons inline in HTML beat icon-font libraries; system fonts (`-apple-system, sans-serif`) beat any web-font download entirely.